

PRACTICAL EXAM

Field	Value
Subject Code	SWE202c
Subject Name	Introduction to Software Engineering
Page No	1
Topic	University Admission & Enrollment System
Time	120 minutes
Tools	MS Word, Draw.io, MS Visio, Astah, Visual Paradigm, StarUML
Note	Open book. Students are NOT allowed to use the Internet.

I. CASE STUDY

EduGate is a fast-growing private higher-education group in Vietnam that operates four member universities and seventeen affiliated colleges across Hanoi, Da Nang, Ho Chi Minh City, and Can Tho. Each year the group receives roughly 320,000 applications from prospective undergraduate and graduate candidates competing for around 38,000 admission slots distributed across more than 240 academic programmes. Until 2024, every member institution ran its own home-grown admission portal, paper-based score-verification process, and manual offer-letter mailing, while a separate Microsoft Access database was used by the group office to consolidate national high-school exam scores downloaded from the Ministry of Education and Training (MOET) data hub. During the July 2024 main admission round the legacy stack failed in three different ways within a single weekend: the public application portal was unreachable for 11 hours under peak traffic, two batches of merit-ranked candidates were emailed offers for the wrong programmes, and 4,700 verified payments were recorded in the bank gateway but never reconciled in the admission database, forcing the group to manually re-issue offers and refund overpayments. The board has therefore approved an enterprise-wide project to build a unified, purpose-built **University Admission & Enrollment System (UAES)**, internally code-named **EduGate Admission Hub**, that must be in production before the May 2026 main admission round.

The system must serve six distinct user groups. Applicants need to register an account using either a Vietnamese citizen ID or a passport number, fill in a multi-step application form (personal information, parental data, high-school transcripts, MOET national-exam scores, language certificates, awards, priority-region codes), upload supporting documents (PDF/JPEG up to 10 MB each), choose up to eight programme preferences in ranked order across the

group's institutions, pay the application fee through e-wallet, credit/debit card, or bank QR transfer, track the real-time status of each application (draft, submitted, under-review, conditional-offer, confirmed, rejected, waitlisted), accept or decline offers within the regulated reply window, and complete enrollment by uploading scanned originals and paying the first-semester tuition deposit. Admission Officers (one team per member institution) must verify uploaded documents, cross-check scores against MOET data, request clarifications, run the merit-ranking algorithm for each programme, generate offer batches, and handle waitlist promotion. Programme Coordinators are faculty members who define each programme's quotas, eligibility rules, additional aptitude tests, and tie-breaking criteria, and approve final admission lists for their programmes. Finance Officers reconcile application-fee and tuition-deposit payments, issue refunds for cancelled or duplicated payments, and generate revenue reports per institution. Customer Support Agents help applicants reset accounts, recover stuck applications, mediate document-rejection disputes, and escalate special-quota cases. Finally, the Group System Administrator is responsible for cross-institution user-and-role management, MOET-API and payment-gateway configuration, admission-round scheduling, audit-log retention, and automated data backups.

The system must satisfy several demanding technical constraints. Because the publication of MOET national-exam results triggers a national rush, the public portal must absorb up to 12,000 concurrent applicants on result-publication day and the form-submission pipeline must sustain at least 800 successful submissions per minute without losing or double-counting a single application. Score-import jobs from the MOET data hub must complete a full 320,000-record refresh within 30 minutes, and the merit-ranking job for any single programme must finish within 5 minutes after officers click "Run ranking". All applicant personal data, scanned identity documents, payment records, and MOET-exam scores must be encrypted in transit (TLS 1.3) and at rest (AES-256), and the system must comply with Vietnam's Personal Data Protection Decree (PDPD 2023) and MOET admission-data retention rules (minimum seven years). Access must be enforced through a role-based access control (RBAC) model covering all six user categories, with per-institution data isolation so that one institution's officers cannot view another institution's applicant pool. The platform must target 99.95% monthly availability during admission rounds, with planned maintenance restricted to the 02:00–03:30 local-time window outside active rounds only. The mobile-first applicant portal must render correctly on screens as narrow as 360 px, support both Vietnamese and English, and pass WCAG 2.1 Level AA standards. The architecture must support horizontal auto-scaling and be geo-distributed across the four campus regions (Hanoi, Da Nang, Ho Chi Minh City, Can Tho) so that additional application nodes can be added transparently behind regional load balancers as the group continues to expand its programme portfolio.

II. QUESTIONS

Answer ALL questions. Base your answers on the **University Admission & Enrollment System (UAES / EduGate Admission Hub)** case study above. Where diagrams are required, use standard UML notation and label all elements clearly.

Question 1: Software Development Model (2.0 points)

1. Identify and justify the most appropriate software development model (SDLC) for the EduGate Admission Hub project. In your justification, address each of the following points: (1.0 points)
 - Whether all requirements are fully known and stable at project inception
 - The need for iterative delivery, regular stakeholder feedback, and early risk mitigation
 - Risk factors specific to UAES (e.g., MOET API integration uncertainty, result-publication-day traffic spikes, merit-ranking algorithm correctness, payment reconciliation, multi-institution data isolation)
 - Team size, projected project duration, and the group's capacity to accommodate evolving requirements as MOET regulations and per-institution admission rules change between rounds
2. Select ONE alternative SDLC model and compare it with your chosen model in the context of UAES. Your comparison must include at least two advantages and two disadvantages for each model. (1.0 points)

Question 2: UC Modeling (1.5 points)

1. Identify FIVE actors in the University Admission & Enrollment System. For each actor, state their primary goal and whether they are a primary or secondary actor. (0.3 points)
2. List at least 6 use cases for UAES. For each use case, state the initiating actor(s) and a one-sentence description of the goal. (0.4 points)
3. Draw a complete UML Use Case Diagram for UAES. The diagram must include: (0.8 points)
 - All identified actors placed correctly inside or outside the system boundary
 - All major use cases inside the system boundary rectangle
 - At least ONE `<<include>>` relationship — provide a brief written rationale
 - At least ONE `<<extend>>` relationship — provide a brief written rationale
 - Any actor-generalisation relationships that apply

Question 3: UC Specification (1.5 points)

Write a complete Use Case Specification for the use case **'Submit Application'** (the applicant completes the multi-step application form, attaches supporting documents, pays the application fee, and receives a submission confirmation). Use the template in the answer sheet and fill in EVERY field. Partial or missing sections will receive partial credit only.

Question 4: Non-Functional Requirements (1.0 points)

1. From the case study, identify and classify at least FOUR non-functional requirements (NFRs) into appropriate quality categories (e.g., Performance, Security, Availability, Usability, Scalability, Maintainability). For each NFR, write one measurable acceptance criterion. Present your answer as a table with the columns: **Category | NFR Description | Acceptance Criterion**. (0.6 points)
2. Identify TWO NFRs from your table that may conflict with each other in the UAES context. Clearly explain why the conflict arises and propose a concrete trade-off strategy that the development team could apply. (0.4 points)

Question 5: Class Diagram (1.5 points)

1. Identify at least SIX key domain classes for UAES. For each class, specify at minimum three attributes (with data types) and two methods. Present this as a structured list or mini class-description table before drawing the diagram. (0.6 points)
2. Draw a complete UML Class Diagram that includes: (0.9 points)
 - All identified classes with their full attribute and method compartments
 - At least ONE inheritance (generalisation) relationship — for example, User as a base class for Applicant, AdmissionOfficer, ProgrammeCoordinator, FinanceOfficer, and SystemAdmin
 - At least TWO association relationships with correct multiplicity labels at both ends
 - At least ONE aggregation OR composition relationship, with correct diamond notation and a written justification
 - At least ONE dependency relationship

Suggested classes (you may add or rename): User, Applicant, AdmissionOfficer, ProgrammeCoordinator, FinanceOfficer, SupportAgent, SystemAdmin, Institution, Programme, AdmissionRound, Application, ProgrammeChoice, Document, ExamScore, Offer, Payment, Refund, Quota, RankingResult, AuditLog, Notification.

Question 6: AI in coding (1.0 points)

This question assesses your ability to use AI coding assistants (e.g., GitHub Copilot, ChatGPT, Claude) as a practical aid during software development. Two scenarios are provided below. For each scenario, you must craft an effective AI prompt and then answer the follow-up tasks.

Important: A good AI prompt must include: (1) clear context about the system, (2) the specific goal, (3) the code or project information itself, and (4) the expected output format.

Scenario — Structure Project Source Code with MVC Architecture

The EduGate Admission Hub back-end is being built with Java Spring Boot. The development team has started the project but all classes (controllers, services, repositories, models, DTOs, and configuration files) are placed in a single package `com.edugate.admission` without any further organisation. The team wants to refactor the project into a clean MVC (Model-View-Controller) architecture so that the codebase is easier to maintain, test, and scale. The current flat package contains the following classes:

```
com.edugate.admission
├─ ApplicationController.java      // handles HTTP requests for application
submission and status
├─ ProgrammeController.java      // handles HTTP requests for programme
catalogue and quotas
├─ OfferController.java          // handles HTTP requests for offer
accept/decline
├─ PaymentController.java        // handles HTTP requests for application
fee and tuition deposit
├─ ApplicationService.java       // business logic for application
lifecycle
├─ RankingService.java           // business logic for merit ranking and
waitlist promotion
├─ OfferService.java             // business logic for offer generation
and reply window
├─ PaymentService.java           // business logic for payment and refund
reconciliation
├─ ApplicationRepository.java     // JPA repository for Application entity
├─ ProgrammeRepository.java      // JPA repository for Programme entity
├─ OfferRepository.java          // JPA repository for Offer entity
├─ PaymentRepository.java        // JPA repository for Payment entity
├─ Application.java              // JPA entity
├─ Programme.java                // JPA entity
├─ Offer.java                    // JPA entity
├─ Payment.java                  // JPA entity
├─ ApplicationDTO.java           // data transfer object
├─ RankingRequestDTO.java        // data transfer object
├─ JwtAuthFilter.java            // Spring Security filter
└─ AppConfig.java                // application-level configuration
```

Your tasks for Scenario:

- **Write an AI Prompt to Restructure the Project.** Write a clear and complete prompt you would submit to an AI assistant to reorganize the flat package above into a proper MVC directory structure. Your prompt must specify: (1) the system context (EduGate Admission Hub, Java Spring Boot), (2) the goal (restructure into MVC with clear package naming), (3) the full list of existing classes provided above, and (4) the expected output format (a

proposed directory tree with package names and brief justification for each layer). (0.4 points)

- **Identify the MVC Layers.** Based on the AI's proposed structure, map each class to its correct MVC layer (Model, View/DTO, Controller, Service, Repository, Config/Security). Present this as a table with columns: **Class Name | MVC Layer | Justification**. (0.4 points)
- **Write a Follow-up AI Prompt.** Write a follow-up prompt asking the AI to generate the refactored package declaration and import statements for `ApplicationController.java` and `RankingService.java`, reflecting the new MVC package structure. (0.2 point)

Question 7: Testing Stage and Type (1.5 points)

1. Map the four standard testing stages to the UAES development process. For each stage, state what is tested, who is responsible, and which testing types are most applicable: (0.5 points)
2. Write THREE test cases for the **'Run Merit Ranking for a Programme'** use case (an admission officer triggers the ranking job for a single programme; the system orders all eligible submitted applications by composite merit score and produces a ranked offer list bounded by the programme quota), covering the scenarios listed below. Present each test case in a table with the columns: **Test Case ID | Test Case Name | Precondition | Test Steps | Expected Result | Testing Type**. (1.0 points)
 - **Happy-path scenario** — the programme has more eligible applicants than its quota, all scores are valid, and the system produces a complete ranked list filling the quota exactly
 - **Boundary/edge-case scenario** — two or more applicants tie on the composite merit score at the last quota slot, and the documented tie-breaking rule must decide who receives the final offer versus the waitlist
 - **Negative/error scenario** — the MOET score data for one or more applicants is missing or marked "pending verification" at the moment of ranking, and the system must reject the run with an actionable error rather than producing a partial ranking

III. SCORING RUBRIC

Q#	Topic	Max Score	Weight	Grader's Score
1	Software Development Model Selection	2.0	20%	
2	UC Modeling	1.5	15%	
3	UC Specification	1.5	15%	
4	Non-Functional Requirements	1.0	10%	
5	Class Diagram	1.5	15%	
6	AI in Coding	1.0	10%	

Q#	Topic	Max Score	Weight	Grader's Score
7	Testing Stage and Type	1.5	15%	
	TOTAL	10.0	100%	

IV. SUBMISSION GUIDELINES

4.1 Submission Format

- The question answers must be in the provided template.
- All work must be submitted as a single .zip file.
- File naming: StudentID_SWE202c.zip (e.g., SE123456_SWE202c.zip)
- The .zip must contain: Answer document in MS Word .docx format.

4.2 Formatting Requirements

- Insert answers below each question. Each question should begin on a new page where possible.
- Clearly label the question number and sub-question before each answer section.
- UML diagrams must be clear, titled, and use correct standard notation.
- Font: Times New Roman or Arial, size 12–13 pt, line spacing 1.5.

4.3 General Evaluation Criteria

- Content accuracy and completeness as required by each question.
- Correct UML syntax, clear and well-annotated diagrams.
- Logical reasoning tied to the specific context of EduGate Admission Hub.
- Clean, professional presentation.

--- END OF EXAM ---

Label all diagrams clearly and justify your design decisions wherever asked.